

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion Book/Guided Tour](#)

[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook](#) › [SubversionBookRemake](#) › [SubversionBook/Preface](#) ›
[SubversionBook/Introduction](#) › [SubversionBook/BasicConcepts](#) › [SubversionBook/GuidedTour](#)

Login:

Password:

[Join](#)

He who has imagination
without learning has wings
but no feet.

[우발적링크](#)
[KLDP사람들일하는곳](#)
[JinLiveCD/LinuxFS](#)
[DrupalHandbookForEndUser](#)

1Chapter. 함께 둘러보기

Table of Contents

- 1.1.
- 1.1. [도움말](#)
- 1.2. [임포트](#)
- 1.3. [리비전: 번호, 키워드, 날짜](#)
 - 1.3.1. [리비전 번호](#)
 - 1.3.2. [리비전 키워드](#)
 - 1.3.3. [리비전 일자](#)
- 1.4. [최초의 체크아웃](#)
- 1.5. [기본적인 작업 사이클](#)
 - 1.5.1. [작업 복사의 갱신](#)
 - 1.5.2. [작업 복사를 변경하기](#)
 - 1.5.3. [자신의 변경점을 조사](#)
 - 1.5.3.1. [svn status](#)
 - 1.5.3.2. [svn diff](#)
 - 1.5.3.3. [svn revert](#)
 - 1.5.4. [충돌의 해소\(다른 사람의 변경점과 Merge\)](#)
 - 1.5.4.1. [충돌을 수동으로 Merge](#)
 - 1.5.4.2. [작업 파일 위에 파일을 카피하기](#)
 - 1.5.4.3. [Punting: svn revert의 이용](#)
 - 1.5.5. [변경점을 커밋](#)
- 1.6. [히스토리 확인하기](#)
 - 1.6.1. [svn log](#)
 - 1.6.2. [svn diff](#)
 - 1.6.2.1. [로컬의 변경 내용을 확인하기](#)
 - 1.6.2.2. [작업 카피와 저장소를 비교하기](#)
 - 1.6.2.3. [저장소와 저장소를 비교하기](#)
 - 1.6.3. [svn cat](#)
 - 1.6.4. [svn list](#)
 - 1.6.5. [히스토리 기능에 대한 마지막 말](#)
- 1.7. [자주 사용되는 다른 명령들](#)
 - 1.7.1. [svn cleanup](#)
 - 1.7.2. [svn import](#)
- 1.8. [요약](#)



kss@kldp.org

1.1.

이제 Subversion을 사용하는 법을 자세히 보도록 하겠습니다. 이 장이 끝날 즈음이면 일상적인 작업에 Subversion을 사용하기 위해 필요한 거의 모든 일들을 할 수 있을 것입니다. 당신의 코드를 처음 체크아웃하는 것으로 시작하여 코드를 수정하고 수정한 내용을 검사합니다. 또 다른 사람이 수정한 내용을 자신의 작업본에 반영하고 검사하며 충돌이 발생할 경우 해결하게 됩니다.

이 장에서 Subversion의 명령어 전부를 열거하지는 않습니다. 그보다는 사용자가 Subversion을 사용하여 가장 보편적으로 하게 될 작업들에 대해 소개를 합니다. 이 장은 당신이 2장>을 읽고 이해했으며 Subversion이 사용하는 일반적인 모델을 알고 있다고 가정합니다. 명령어의 전체 레퍼런스는 8장 >을 보십시오.

1.1. 도움말

계속 진행하기 전에 가장 중요한 Subversion 명령어인 **svn help** 에 대해 알아 보겠습니다. Subversion 커맨드 라인 클라이언트는 스스로 문서를 생성하여 출력합니다. 언제라도 **svn help** 서브 커맨드 라고 치면 서브 커맨드의 문법, 옵션 스위치, 동작을 알려 줍니다.

1.2. импорт

svn import 명령으로 Subversion의 저장소(repository)에 새로운 프로젝트를 импорт 할 수 있습니다. Subversion 서버를 설정할 때 최초로 실행하게 될 커맨드이지만 매우 빈번하게 사용되는 것은 아닙니다. import 의 상세한 기능에 대하여는 이 장의 뒤 쪽에 있는 "svn import" 섹션 >을 보십시오.

1.3. 리비전: 번호, 키워드, 날짜

진행하기 전에 저장소 내에 특정한 리비전을 지칭하는 방법에 대해 알아 둘 필요가 있습니다. >에서 본 것처럼 리비전은 특정한 순간 저장소(repository)의 "순간포착 사진(snapshot)"입니다. 커밋을 반복하여 저장소가 커짐에 따라 이 스냅샷들을 구분할 방법이 필요하게 됩니다.

--revision (-r) 옵션 뒤에 원하는 리비전 번호를 적어 (**svn --revision REV**) 리비전을 명시할 수 있습니다. 또는 두 리비전을 콜론으로 구분하여 (**svn --revision REV1:REV2**). 범위를 지정할 수도 있습니다. 리비전은 번호, 키워드, 날짜 세가지 방법으로 참조할 수 있습니다.

1.3.1. 리비전 번호

새로운 Subversion 저장소를 만들면 리비전 0부터 시작하게 됩니다. 이후 커밋을 실행할 때마다 리비전 번호는 1씩 증가합니다. 커밋이 완료될 때마다 Subversion 클라이언트는 아래와 같이 새로운 리비전 번호를 알려줍니다.

```
$ svn commit --message "Corrected number of cheese slices. "
Sending          sandwich.txt
Transmitting file data .
Committed revision 3.
```

나중에 이 리비전을 참조하고 싶을 때 (이 장의 뒤에서 어떤 경우에 그런 일이 필요한가를 설명합니다) "3"으로 참조할 수 있습니다.

1.3.2. 리비전 키워드

Subversion 클라이언트는 다양한 *리비전 키워드*를 이해할 수 있습니다. 이러한 키워드는 --revision 옵션 뒤에 숫자값 대신에 사용할 수 있으며 Subversion에 의해 특정의 리비전 번호로 변환됩니다:

Note: 작업본의 모든 디렉토리에는 .svn이라는 이름의 관리 영역이 있습니다. Subversion은 디렉토리 안의 각 파일의 사본을 이 관리 영역에 보관합니다. 이 사본은 사용자가 자신의 작업본을 갱신(update)한 가장 최근의 리비전 (이것을"BASE"리비전이라고 합니다)에 있던 파일을 수정 없이 (키워드 확장, end-of-line변환을 비롯하여 어떠한 일도 수행하지 않고) 보관하는 것입니다. 이 파일을 "원래의 작업복사 (pristine copy)"라고 말합니다.

HEAD

저장소(repository)에 있는 최신 리비전입니다.

BASE

작업 복사에 있는 파일 디렉토리의 "원래의 작업복사"리비전입니다.

COMMITTED

BASE 이전 또는 BASE 에서 변경된 아이템이 있는 마지막 리비전입니다.

PREV

변경이 있던 마지막 리비전 **직전**의 리비전입니다. (즉 COMMITTED - 1 이 됩니다)

커맨드 실행시의 리비전 키워드의 예입니다 (아직 커맨드의 의미를 몰라도 괜찮습니다. 이 장을 진행하면서 설명합니다):

```
$ svn diff --revision PREV:COMMITTED foo.c
# foo.c 에 커밋한 마지막 변경을 표시
```

```
$ svn log --revision HEAD
```

```
# 저장소에 마지막으로 커밋할 때의 로그 메시지를 표시

$ svn diff --revision HEAD
# 작업 카피내 파일(로컬에서 수정되었을 수 있음)을 저장소의
# 최신 버전과 비교

$ svn diff --revision BASE:HEAD foo.c
# 작업 카피의 "수정원" foo.c(로컬에서 수정한 것이 반영되지 않음)를
# 저장소의 최신 버전과 비교

$ svn log --revision BASE:HEAD
# 마지막으로 업데이트한 이후의 커밋된 로그를 모두 표시

$ svn update --revision PREV foo.c
# foo.c 의 마지막 변경을 되돌린다
# (foo.c 의 작업 리비전이 감소함)
```

이러한 키워드를 이용하면 작업본이나 특정 리비전을 헛갈리는 번호가 아닌 키워드로 지정하여 자주 쓰는 여러 유용한 명령을 수행할 수 있습니다.

1.3.3. 리비전 일자

리비전 번호나 리비전 키워드를 사용할 수 있는 경우라면 언제나 날짜와 시각도 중괄호 "{}" 안에 넣어 리비전을 지정할 수 있습니다. 날짜와 리비전 번호를 동시에 사용해 범위를 지정할 수도 있습니다.

Subversion은 매우 많은 일자 형식을 받아들일 수가 있습니다. 일자 형식이 공백을 포함한 경우에는 반드시 인용부호로 묶어 주어야 합니다. 여기에서는 Subversion이 받아들일 수 있는 형식 중 몇 가지 예를 보여줍니다.

```
$ svn checkout --revision {2002-02-17}
$ svn checkout --revision {2/17/02}
$ svn checkout --revision {"17 Feb"}
$ svn checkout --revision {"17 Feb 2002"}
$ svn checkout --revision {"17 Feb 2002 15:30"}
$ svn checkout --revision {"17 Feb 2002 15:30:12 GMT"}
$ svn checkout --revision {"10 days ago"}
$ svn checkout --revision {"last week"}
$ svn checkout --revision {"yesterday"}
```

리비전에 날짜를 지정한 경우 Subversion은 그 날짜 이전의 가장 최신 리비전을 찾아내려고 합니다.

```
$ svn log --revision {11/28/2002}
```

```
-----
r12 | ira | 2002-11-27 12:31:51 -0600 (Wed, 27 Nov 2002) | 6 lines
```

Subversion은 하루가 빨라요?

리비전으로 시각을 지정하지 않고 날짜만 지정했다면 (예를 들어 11/27/02) 사용자는 11월 27일에 추가된 리비전 중 마지막 리비전을 다루어야 한다고 생각할지도 모릅니다. 그러나 실제로는 26일 또는 그 이전의 리비전을 얻게 됩니다. Subversion은 지정된 일시 **이전의 저장소 중 최신의 리비전**을 찾아내려고 한다는 것에 유의하십시오. 11/27/02와 같이 시각 없이 날짜만을 지정하면 Subversion은 시각 00:00:00가 지정되었다고 가정하고 결과적으로 27일날 커밋된 것은 고려하지 않습니다.

27일 분을 같이 검색하고 싶다면 시각을 같이 지정하거나 ("27 Nov 2002 23:59") 아니면 간단히 다음날을 지정하면 됩니다. ("28 Nov 2002")

날짜 범위를 사용할 수도 있습니다. Subversion은 범위 안에 있는 모든 리비전을 대상으로 검색합니다. 지정된 두 날짜는 검색에 포함됩니다.

```
$ svn log --revision {2002-11-20}:{2002-11-29}
```

앞서 지정한 것처럼 날짜와 리비전 번호를 같이 사용할 수도 있습니다.

```
$ svn log -r {11/20/02}:4040
```

1.4. 최초의 체크아웃

대부분의 경우 저장소로부터 프로젝트를 *체크아웃(checkout)* 하는 것으로 Subversion을 시작합니다. 저장소를 "체크아웃"하면 사용자의 로컬 머신에는 저장소의 사본이 생성됩니다. 이 사본에는 커멘트 라인에서 지정한 Subversion 저장소의 HEAD(최신 리비전)가 저장됩니다.

```
$ svn checkout http://svn.collab.net/repos/svn/trunk
A trunk/subversion.dsw
A trunk/svn_check.dsp
A trunk/COMMITTERS
A trunk/configure.in
A trunk/IDEAS
```

Checked out revision 2499.

저장소의 레이아웃

위의 URL에서 trunk가 무엇인지 궁금할 것입니다. 그것은 사용자가 Subversion 저장소를 배치할 때 권장되는 방법으로 4장 "Branching and Merging"(>)에서 자세히 다룹니다.

위의 예는 trunk 디렉토리의 체크아웃이었지만 체크아웃 URL에 서브 디렉토리를 지정하면 더 깊은 계층에 있는 하위 디렉토리도 간단하게 체크아웃 할 수 있습니다.

```
$ svn checkout http://svn.collab.net/repos/svn/trunk/doc/book/tools
A tools/readme-dblite.html
A tools/fo-stylesheet.xsl
A tools/svnbook.el
A tools/dtd
A tools/dtd/dblite.dtd
```

Checked out revision 3678.

Subversion은 "잠금·수정·잠금 해제" 모델이 아니라 "복사·수정·합침(Merge)" 모델을 사용하므로 (>) 체크아웃하면 바로 가져온 파일과 디렉토리를 수정할 수 있습니다. (이렇게 가져온 파일과 디렉토리의 모임을 *작업 복사*라고 합니다).

바꾸어 말하면 여러분의 "작업 복사"는 시스템에 있는 다른 파일이나 디렉토리들과 다를 것이 없습니다. [\[1\]](#) 편집하거나 변경하거나 이동하거나 작업 카피 전체를 통째로 삭제해버릴 수도 있습니다.

Note: 작업본은 시스템에 있는 "다른 파일이나 디렉토리들과 아무런 차이점은 없습니다"만 작업 카피에 속하는 파일이나 디렉토리의 위치나 이름을 다시 편성했을 경우에는 항상 Subversion에 그것을 알려주어야 합니다. 만약 작업 복사에 속하는 파일이나 디렉토리를 복사하거나 이동하고 싶은 경우에는 운영체제에서 제공하는 복사나 이동 명령어 대신에 **svn copy**와 **svn move**를 사용해 주세요. 이 장의 뒤쪽에서 좀 더 자세하게 설명합니다.

파일을 고쳤거나, 또는 새로운 파일을 만들거나, 디렉토리를 옮기는 등의 변경사항은 다음 커밋할 때까지 Subversion 서버에 따로 알릴 필요가 전혀 없습니다.

. svn 디렉토리는 무엇입니까?

작업 카피에 속하는 모든 디렉토리에는 `.svn`이라는 *관리 영역*이 있습니다. 보통 파일 목록을 표시하는 명령(`ls`, `dir`)은 이 디렉토리를 표시하지 않습니다. 그렇지만 이 디렉토리는 매우 중요합니다. 무슨 일을 하더라도 관리 영역을 지우거나 변경하지 마십시오! Subversion은 작업 복사를 관리하는데 이 디렉토리를 사용합니다.

저장소(repository)의 URL만 인수로 줘서 작업 카피를 체크아웃해도 되지만 저장소(repository)의 URL 뒤에 디렉토리를 따로 지정하여 체크아웃할 수도 있습니다. 이 경우 지정한 디렉토리안에 작업 카피가 만들어집니다. 예를 들어:

```
$ svn checkout http://svn.collab.net/repos/svn/trunk subv
A subv/subversion.dsw
A subv/svn_check.dsp
A subv/COMMITTERS
A subv/configure.in
A subv/IDEAS
```

Checked out revision 2499.

이렇게 하면 우리가 앞에서 했던 것처럼 `trunk`라는 이름의 디렉토리 대신에 `subv`이라는 디렉토리에 작업 카피가 만들어 집니다.

1.5. 기본적인 작업 사이클

Subversion에는 많은 기능이 있지만 자주 사용하는 기능은 몇 가지 뿐입니다. 이 장에서는 제일 자주 쓰이는 것들을 설명합니다.

전형적인 작업 사이클은 다음과 같습니다.

- 작업 복사의 업데이트
 - **svn update**
- 변경
 - **svn add**
 - **svn delete**
 - **svn move**
 - **svn copy**
- 자신의 변경점 확인
 - **svn status**
 - **svn diff**
 - **svn revert**
- 다른 사람의 변경과 Merge
 - **svn merge**
 - **svn resolved**
- 자신의 변경점을 커밋
 - **svn commit**

1.5.1. 작업 복사의 갱신

팀을 만들어 작업하고 있는 프로젝트에서는 자신의 작업 복사를 *갱신*하고 싶을때가 있을 것입니다. 즉 다른 멤버의 변경점을 모두 받는 것이지요. 이 경우 **svn update**로 자신의 작업 복사를 저장소(repository)의 최신 버전에 맞춥니다.

```
$ svn update
U . /foo.c
U . /bar.c
Updated to revision 2.
```

이 경우 당신이 마지막으로 작업 복사를 갱신하고 나서 다른 사람이 `foo.c` 와 `bar.c` 에 가한 변경을 커밋했고 Subversion은 이 변경을 당신의 작업 복사에 포함시키기 위해서 두 파일을 갱신했습니다.

svn update 의 출력을 좀 더 자세하게 봅시다. 서버가 변경점을 작업 복사에 보낼 때 문자 코드가 각각의 파일 의 옆에 표시되어 당신의 작업 카피를 최신으로 하기 위해서 어떠한 동작을 했는지를 알려줍니다:

U foo

파일 foo 는 갱신되었습니다(서버로부터 변경을 받아들였습니다).

A foo

파일이나 디렉토리 foo 는 당신의 작업 복사에 추가되었습니다.

D foo

파일이나 디렉토리인 foo 는 당신의 작업 복사로부터 삭제되었습니다.

R foo

파일이나 디렉토리인 foo 는 치환되었습니다. 즉 foo 는 삭제되어 같은 이름의 새로운 파일 또는 디렉토리가 추가되었습니다. 양쪽 모두는 같은 이름이지만 저장소(repository)는 그것들을 다른 히스토리를 가진 다른 개체(object)로 간주합니다.

G foo

파일 foo 는 새로운 변경점을 저장소(repository)로부터 받았습니다만 그 파일은 로컬 카피에서도 수정이 있었습니다. 그러나 양쪽의 수정은 겹치지 않기 때문에 Subversion 이 저장소에서 온 변경점을 로컬 카피에 문제없이 합쳤습니다(merge).

C foo

파일 foo는 서버로부터 충돌이 있는 변경을 받았습니다. 서버로부터의 변경은 당신 자신의 변경과 직접 겹치고 있습니다. 그렇지만 걱정할 필요는 없습니다. 이 충돌은 사람(즉 당신)이 해소하지 않으면 안됩니다. 이 장의 다음에 이 상황에 대해 논의합니다.

1.5.2. 작업 복사를 변경하기

이제 자신의 작업 복사로 변경을 더할 수가 있습니다. 이하와 같은 비교적 특수한 변경을 할 수도 있습니다. 새로운 기능 을 추가하거나 버그를 고치거나 등입니다. 이러한 경우에 사용하는 Subversion 커맨드는 **svn add**, **svn delete**, **svn copy**, **svn move** 등입니다. 그러나 이미 Subversion의 관리하에 있는 파일을 단지 편집할 뿐이라면 커밋하기까지는 그러한 커맨드를 사용할 필요는 없습니다. 여러분이 작업 복사에 가할 수 있는 변경은 두 가지가 있습니다.

파일의 변경

이것은 제일 단순한 종류의 변경입니다. 파일을 변경하는 것에 대하여 Subversion에 보고할 필요는 없습니다. 어느 파일이 변경되었는지는 Subversion이 자동적으로 검출할 수가 있습니다.

트리의 변경

Subversion에 대해, 삭제, 추가, 카피, 이동이 예약되어 있다고 파일이나 디렉토리를 "표시"하도록 의뢰할 수가 있습니다. 이러한 변경은 작업 카피상에서는 즉시 일어납니다만 다음에 당신이 커밋할 때까지 저장소(repository) 상에서는 추가 혹은 삭제는 일어나지 않습니다.

파일을 변경하려면 텍스트 문자 편집기, 워드 프로세서, 그래픽 프로그램 등 어떤 도구라도 사용할 수가 있습니다. Subversion은 바이너리 파일도 텍스트 파일을 취급하는 것과 같이 쉽게 또 충분히 효율적으로 다룰 수 있습니다.

여기에서는 트리의 변경하기 위해 제일 자주 이용되는 네 개의 Subversion 서브커맨드를 개관해 둡니다 (**svn import** 와 **svn mkdir** 는 나중에 다룰 것입니다).

svn add foo

foo 를 저장소(repository)에 추가하도록 예약해 둡니다. 다음 번 커밋할 때 foo 는 부모 디렉토리의 자식이 됩니다. foo가 디렉토리인 경우 foo 아래에 있는 모든 파일 역시 추가 예고의 대상이 됩니다. foo 만을 추가하고 싶은 경우는--non-recursive (-N) 옵션을 지정해 주세요.

svn delete foo

foo 를 저장소(repository)로부터 삭제도록 예약 합니다. foo 가 파일이면 작업 복사로부터 즉시 삭제됩니다. foo가 디렉토리이면 바로 삭제되지 않고 삭제 예약 상태로 설정됩니다. 변경을 커밋하면 foo 는 작업 복사와 저장소(repository)로부터 삭제됩니다. [\[2\]](#)

svn copy foo bar

foo를 복사해서 새로운 아이템 bar를 만듭니다. bar는 자동적으로 추가 예고됩니다. bar가 다음의 커밋으로 저장소(repository)에 추가될 때 복사의 히스토리가 기록됩니다(foo의 복사본이라는 히스토리).

svn move foo bar

이 명령은 **svn copy foo bar**; **svn delete foo** 를 실행하는 것과 완전히 같습니다. 즉 bar 는foo의 복사본으로서 추가 예약되고 foo 는 삭제 예약됩니다.

작업 복사없이 저장소(repository)를 변경하는 것

이 장(chapter)의 처음에 변경 사항을 저장소(repository)에 반영시키려면 커밋할 필요가 있다고 했습니다. 그것이 완벽하게 사실이라고 할 수는 없습니다. 저장소(repository)에 대해서 트리의 변경을 직접 커밋하는 것 같은 몇개의 커멘드도 **있습니다**. 이것은 서브커멘드가 작업 복사 패스가 아니고 직접 URL을 조작하는 경우에만 일어납니다. 특히 **svn mkdir**, **svn copy**, **svn move**, **svn delete**의 특수한 이용은 URL을 직접 조작할 수 있습니다.

URL의 조작이 그러한 방법으로 행동하는 것은 작업 카피에 대한 조작 커멘드가 저장소(repository)에 커밋하려는 변경점을 세트해 두는 어떤 종류의 "준비 영역"으로서 작업 복사를 사용하기 때문입니다. URL을 조작하는 커멘드는 그런 여유가 없기 때문에 직접 URL을 조작하면 위에서 말한 작업들은 모두 즉시 커밋하는 효과를 냅니다.

1.5.3. 자신의 변경점을 조사

변경이 완료하면 저장소(repository)에 커밋할 필요가 있습니다. 하지만 보통 커밋하기 전에 자신이 무엇을 변경했는지 정확하게 봐 두는 게 좋습니다. 커밋 전에 변경점을 확인함으로써 보다 정확한 로그 메시지를 붙이는 것이 가능할 뿐만 아니라 불충분하게 수정된 것을 발견할 수도 있고 커밋하기 전에 그 변경을 파기하거나 할 기회가 주어집니다. 게다가 공개하기 전에 변경점을 재검토하거나 자세하게 조사할 좋은 기회이기도 합니다. **svn status**, **svn diff**, **svn revert** 를 사용해 여러분이 어떤 변경을 했는지를 정확하게 볼 수 있습니다. 여러분은 보통 앞쪽 두 커멘드로 작업 카피중의 어느 파일을 변경했는지를 조사하고 세번째 커멘드로 그 중의 몇몇(혹은 전부)의 변경을 취소하게 될 것입니다.

Subversion은 이 작업을 하기 위해서 최적화되어 왔고 저장소와 통신하지 않고도 많은 일을 할 수 있습니다. 특히 작업 복사에는 . svn 라고 하는 숨은 디렉토리가 있어 여기에 작업 복사의 "원본 리비전(prestine)"의 카피가 있습니다. 이것을 잘 사용해 Subversion은 당신의 작업 파일의 어떤 것이 변경되었는지를 재빠르게 알 수가 있고 저장소(repository)와 통신하지 않고도 변경한 것을 되돌릴 수 있습니다.

1.5.3.1. svn status

아마 어느 Subversion 커멘드보다**svn status** 커멘드를 자주 이용하게 될 것입니다.

CVS 사용자에게: Update를 멈추세요!

작업 복사에 어떠한 변경을 행했는지를 확인하기 위해서 아마 여러분은 **cvs update**를 사용하고 있겠지요. **svn status**는 작업 복사에 행해진 변경에 대해 모든 필요한 정보를 제공해 줍니다. 게다가 저장소(repository)에 액세스 하지도 않고 다른 사람이 행한 변경이 받아들여질 가능성도 없습니다.

Subversion에서 **update**는 마지막 갱신 후에 저장소(repository)에 커밋된 모든 변경을 작업 카피에 반영할 뿐입니다. 로컬 카피에 대해서 행한 변경을 확인하기 위해서 **update** 를 사용하는 버릇을 고쳐야 합니다.

자신의 작업 카피 최상위에서 인수없이 **svn status** 를 실행하면 모든 파일과 트리에 대한 모든 수정

사항을 검출할 수 있습니다. 이 예는 **svn status** 가 돌려줄 수 있는 모든 다른 상태 코드를 볼 수가 있도록 만든 것입니다. #의 뒤에 써 있는 텍스트는 **svn status**가 출력한 것이 아닙니다.

```
$ svn status
L      . /abc.c                # abc.c에 대한 lock을 .svn 디렉토리에 갖고 있습니다.
M      . /bar.c                # bar.c는 수정되었습니다.
M      . /baz.c                # baz.c has property but no content modifications
?      . /foo.o                # svn(Subversion)은 foo.o를 관리하지 않습니다.
!      . /some_dir             # svn이 관리하기는 하지만 없어졌거나 불완전합니다.
~      . /qux                  # 디렉토리라고 기록되어있지만 파일이거나 반대의 경우입니다.
A +    . /moved_dir            # 어디에서 온 파일인지에 대한 기록과 함께 추가되었습니다.
M +    . /moved_dir/README     # 기록과 함께 추가되었고 로컬에서 수정되었습니다.
D      . /stuff/fish.c         # 삭제되기로 예약되었습니다.
A      . /stuff/loot/blow.h     # 추가되기로 예약되었습니다.
C      . /stuff/loot/lump.c     # 충돌이 있습니다.
      S . /stuff/squawk         # 이 파일이나 디렉토리는 브랜치(branch)로 전환되었습니다.
```

이 출력 형식에서 **svn status**는 다섯 개의 문자를 표시하고 그 후에 몇개의 공백이 표시되며 파일 또는 디렉토리 이름이 그 후에 표시되고 있습니다. 첫번째 문자는 파일 또는 디렉토리와 그 내용에 대한 상태를 나타내고 있습니다. 여기서 표시되는 코드는 다음과 같습니다.

A file_or_dir

파일 또는 디렉토리 file_or_dir 은 저장소(repository)에 추가 예약되었습니다.

M file

file의 내용은 변경되었습니다.

D file_or_dir

파일 또는 디렉토리 file_or_dir은 저장소(repository)로에서 삭제 예약되었습니다.

X dir

디렉토리 dir은 버전화 되지 않았지만 Subversion의 외부 정의에 관련되어 있습니다. 외부 정의에 대해 자세한 내용은>를 보세요.

? file_or_dir

파일 또는 디렉토리 file_or_dir은 버전 관리하에는 없습니다. --quiet (또는 -q) 옵션을 **svn status** 에 건네주거나 부모 디렉토리에 svn:ignore 속성을 설정하면 물음표를 표시하지 않습니다. 무시되는 파일에 대해 자세한 정보는 >를 보세요.

! file_or_dir

파일 또는 디렉토리 file_or_dir은 버전 관리하에 있지만 없어졌거나 불완전한 상태에 있습니다. Subversion 이외의 커맨드를 사용해서 삭제했을 경우 에 그 아이템은 없어질 수 있습니다. 디렉토리의 경우에 체크아웃이나 갱신이 도중에 중단되었다면 불완전한 상태가 될 수 있습니다. **svn update**를 사용하면 곧바로 저장소(repository)로부터 파일 또는 디렉토리를 한번 더 꺼낼 수가 있습니다. **svn revert file**를 사용하면 없어진 파일을 복원할 수가 있습니다.

~ file_or_dir

파일 또는 디렉토리 file_or_dir은 작업 복사와 저장소에 서로 다른 타입의 개체(object)로 존재하고 있습니다. 예를 들어 저장소(repository)에 있는 것은 파일이지만 **svn delete** 나 **svn add**를 사용하지 않고 작업 복사에서 대응하는 파일을 삭제한 후 같은 이름의 디렉토리를 만든 경우입니다.

C file

file_or_dir는 충돌 상태에 있습니다. 즉 갱신 처리중에 서버로부터 받은 변경은 작업 복사에 당신이 한 변경 부분과 겹치고 있습니다. 이 경우 저장소(repository)로 변경을 커밋하기 전에 충돌 문제를 해결해야 합니다.

두번째 문자는 파일 또는 디렉토리의 속성을 나타내고 있습니다 (자세하게는 참조하세요). 만약 M이 표시되었으면 속성이 수정된 것을 나타내고 있습니다. 그렇지 않으면 공백이 표시됩니다.

세번째의 문자는 빈칸과 L 둘 중 하나가 표시되며 L◆ 표시되는 경우는 Subversion이 그 파일(혹은 디렉토리)를 .svn 중에 잠그고 있는 것을 의미합니다. **svn commit**가 실행되고 있는 도중에 **svn status**를 실행하면 L이 표시됩니다. 아마 당신은 로그 메시지를 변경하고 있을 것입니다. Subversion

이 실행중이지 않다면 아마 Subversion이 실행중에 중단되어서 잠금을 해제하지 못했을 것입니다. **svn cleanup**을 실행하여 잠금을 해제해야 합니다(이 장의 뒷부분에서 좀 더 다룹니다).

네번째의 문자는 공백이나 +가 표시되며 추가 또는 수정되어 그것이 히스토리에 추가 예약되어 있는 것을 의미합니다. 이것은 파일이나 디렉토리에 대해 **svn move**나 **svn copy**를 했을 때에 잘 일어납니다. A+의 표시가 있는 경우 그 아이템은 히스토리 포함의 추가 예약되어 있는 것을 의미합니다. 그것은 파일이거나 복사된 디렉토리의 루트입니다. +는 그 아이템이 히스토리에 추가 예약된 서브 트리의 일부인 것을 의미합니다. 즉 그 아이템의 부모가 카피되어 커밋을 기다리고 있습니다. M+ 는 아이템이 히스토리에 추가 예고된 서브 트리의 일부이며 **한편** 로컬에서 변경된 적이 있다는 것입니다. 커밋할 때 가장 처음으로 부모가 히스토리와 함께 추가됩니다(복사됩니다). 그 의미는 이 파일은 카피에 자동적으로 존재한다고 하는 것입니다. 그 다음에 로컬의 변경점은 카피에 업로드됩니다.

다섯번째 문자는 공백이나 S가 표시됩니다. 파일이나 디렉토리는 작업 복사의 나머지 경로(path)로부터 (**svn switch** 명령에 의해) 브랜치로 바뀐 것을 의미합니다.

svn status에 경로(path)를 지정하면 그 경로에 대한 정보만을 표시합니다.

```
$ svn status stuff/fish.c
D      stuff/fish.c
```

svn status도 --verbose (-v) 옵션을 줄 수 있습니다. 그 경우 작업 복사의 (변경되지 않은 파일도 포함하여) **모든** 아이템의 상태를 표시합니다.

```
$ svn status --verbose
M      44      23    sally    . /README
      44      30    sally    . /INSTALL
M      44      20    harry    . /bar.c
      44      18    ira      . /stuff
      44      35    harry    . /stuff/trout.c
D      44      19    ira      . /stuff/fish.c
      44      21    sally    . /stuff/things
A      0       ?     ?       . /stuff/things/bloo.h
      44      36    harry    . /stuff/things/gloo.c
```

이것은 **svn status**의 "긴 표시 형식" 출력입니다. 첫번째 열(column)은 같습니다만 두번째 열은 아이템의 작업 리비전입니다. 세번째와 네번째는 각각 아이템이 마지막에 변경된 리비전과 누가 변경했는지 표시합니다.

지금까지 나온 **svn status**의 실행은 모두 저장소(repository)와 통신을 하지 않습니다. 그것은 단지 작업 복사안에서 .svn 디렉토리의 메타데이터를 비교함으로써 로컬 머신상에서만 동작합니다. 마지막으로 --show-updates(-u) 옵션이 있습니다. 이것은 저장소(repository)와 통신해서 **최신보다 낡은** (out of date) 상태에 관계한 정보를 추가 표시합니다.

```
$ svn status --show-updates --verbose
M      *      44      23    sally    . /README
M      *      44      20    harry    . /bar.c
      *      44      35    harry    . /stuff/trout.c
D      44      19    ira      . /stuff/fish.c
A      0       ?     ?       . /stuff/things/bloo.h
```

두 별표에 주의하세요. 이 상태로 **svn update**를 실행하면 README와 trout.c 의 변경점을 받게 됩니다. 이것은 매우 도움이 되는 정보입니다. 당신은 커밋하기 전에 README에 관한 서버상의 변경점을 받아와서 갱신해야 합니다. 그렇지 않으면 최신이 아니라는 이유로 커밋은 실패하겠지요(자세하게는 다음에 말합니다).

1.5.3.2. svn diff

자신의 변경점을 조사하는 다른 방법은 **svn diff** 명령을 사용하는 것입니다. **svn diff**를 인수 없이 실행하면 자신이 어떤 변경을 했는지 **정확하게** 알 수가 있습니다. 이 때의 출력 형식은 unified diff 형식입니다: [\[3\]](#)

```
$ svn diff
Index: . /bar.c
=====
--- . /bar.c
+++ . /bar.c    Mon Jul 15 17:58:18 2002
@@ -1, 7 +1, 12 @@
+#include sys/types.h
+#include sys/stat.h
+#include unistd.h
+
+#include stdio.h

int main(void) {
- printf("Sixty-four slices of American Cheese...\n");
+ printf("Sixty-five slices of American Cheese...\n");
return 0;
}

Index: . /README
=====
--- . /README
+++ . /README    Mon Jul 15 17:58:18 2002
@@ -193, 3 +193, 4 @@
+Note to self: pick up laundry.

Index: . /stuff/fish.c
=====
--- . /stuff/fish.c
+++ . /stuff/fish.c    Mon Jul 15 17:58:18 2002
-Welcome to the file known as 'fish'.
-Information on fish will be here soon.

Index: . /stuff/things/bloo.h
=====
--- . /stuff/things/bloo.h
+++ . /stuff/things/bloo.h    Mon Jul 15 17:58:18 2002
+Here is a new file to describe
+things about bloo.
```

svn diff 명령은 .svn 에 있는 "수정원 리비전(pristine)"과 작업 복사 안의 파일을 비교한 결과를 출력합니다. 추가 예약된 파일은 모두 추가된 텍스트로서 표시되고 삭제 예고되고 있는 파일은 모두 삭제된 텍스트로 표시됩니다.

출력은 *unified diff* 형식으로 표시됩니다. 즉 삭제된 줄은 앞에 -가 표시되고 추가된 줄은 앞에 +가 표시됩니다. **svn diff**는 **patch** 프로그램에 유용하도록 파일 이름과 오프셋(offset) 정보를 표시합니다. 이 때문에 diff의 출력을 파일에 쓰는 것으로 "패치"를 생성할 수가 있습니다:

```
$ svn diff > patchfile
```

예를 들어 패치 파일을 다른 개발자에게 보내 커밋전에 재검토나 테스트를 할 수가 있습니다.

1.5.3.3. svn revert

위의 diff 출력을 보고 README을 잘못 수정한 것을 알았다고 합시다.

이것은 **svn revert**를 사용할 수 있는 매우 좋은 기회입니다.

```
$ svn revert README
Reverted . /README
```

Subversion은 그 파일을 .svn에 있는 "수정원 리비전"에 있는 것으로 덮어써서 수정하기 전으로 되돌립니다. 그러나 **svn revert**가 **어떠한** 예약 조작도 취소할 수가 있는데 주의하세요. 예를 들어 최종적

으로 새로운 파일을 추가하는 것을 그만둘 수가 있습니다.

```
$ svn status foo
?      foo

$ svn add foo
A      foo

$ svn revert foo
Reverted foo

$ svn status foo
?      foo
```

Note: `svn revert /ITEM`은 작업 카피로부터 `/ITEM`을 삭제하고 `svn update /ITEM`을 실행한 것과 완전히 같은 효과가 있습니다. 그러나 만약 파일을 기초로 되돌린다면 `svn revert`에는 한 가지 중요한 차이가 있는데 그것은 파일을 바탕으로 되돌리기 위해서 저장소와 통신할 필요가 없다는 것입니다.

혹은 실수로 버전 관리로부터 파일을 지워버렸을지도 모릅니다.

```
$ svn status README
      README

$ svn delete README
D      README

$ svn revert README
Reverted README

$ svn status README
      README
```

엄마 네트워크가 없어도 돼!

지금까지 봐 온 세 개의 명령(`svn status`, `svn diff`, `svn revert`)는 네트워크에 대한 액세스없이 실행할 수 있습니다. 이것으로 비행기 여행중이나 통근 전철을 타고 있을 때 해변에서 컴퓨터를 쓸 때처럼 네트워크에 접속되어 있지 않은 장소에서도 수정 작업을 계속 관리할 수 있습니다.

이것은 Subversion이 `.svn` 관리 영역에 수정된 리비전 파일의 캐쉬를 보존함으로써 가능합니다. 이것으로 Subversion은 **네트워크에 접속하지 않고** 파일에 관한 보고나 수정사항의 취소를 할 수 있습니다. 또 이 캐쉬("text-base"로 불립니다)는 로컬 수정을 서버에 커밋할 때에 수정된 버전과의 차이점만을 압축해서 보낼 수 있도록 합니다. 이 캐쉬를 가지고 있는 것은 매우 큰 이익이 됩니다. 빠른 네트워크 환경이라고해도 파일 전체를 전송 하는 것보다 차이점만을 보내는 편이 훨씬 빠를 것입니다. 처음에는 그렇게 중요한 일처럼 생각되지 않을지도 모르지만 만약 파일을 전부 전송해야 한다면 400 MB의 파일을 한 줄만 바꿨는데도 커밋할 때 파일 전체를 서버에 전송해야 한다고 생각해보세요!

1.5.4. 충돌의 해소(다른 사람의 변경점과 Merge)

이제 `svn status -u`가 어떻게 충돌을 예고할 수 있는지를 알고 있습니다. `svn update`를 실행했더니 재미있는 일이 일어났다고 합니다.

```
$ svn update
U  . /INSTALL
G  . /README
C  . /bar.c
```

U와 G로 표시된 파일은 특별히 생각할 것이 없습니다. 이 둘은 저장소(repository)로부터의 변경을 잘 받아들였습니다. U로 표시된 파일은 로컬에서는 어떤 변경도 없었습니다. 따라서 저장소로부터의 변경 사항을 받아서 갱신(Updated)되었습니다. G는 합쳐졌음(merged)을 의미합니다. 즉 파일이 로컬

에서 변경되었지만 저장소에서의 변경 부분과는 전혀 겹치지 않았습니다.

그러나 C는 충돌을 나타냅니다. 이것은 서버로부터의 변경 사항이 당신의 변경사항과 겹치는 것을 의미하며 당신은 어느 쪽인지를 직접 선택해야 합니다.

만약 충돌이 일어난다면 Subversion 클라이언트는 다음의 세 가지 작업을 수행합니다:

- Subversion은 갱신(update) 처리중에 C를 표시해 그 파일이 "충돌이 있음"을 알립니다.
- Subversion은 *충돌 표지*를 겹치고 있는 장소에 두어 충돌이 일어난 내용을 보고 알 수 있도록 합니다.
- 충돌이 있는 파일이름의 뒤에 특정한 표시(문자를 덧붙임)를 각각 붙인 세 개의 추가 파일을 작업 복사에 만듭니다.

filename.mine

작업 복사를 갱신하기 전에 작업 복사에 있던 파일입니다. 즉 충돌 표지(어떤 부분에서 충돌이 있었는지를 보여주는)를 포함하고 있지 않습니다. 이 파일은 당신이 한 마지막 변경이 포함되어 있습니다.

filename.rOLDREV

이것은 작업 복사를 갱신하기 전의 BASE(.svn에 저장되어 있는) 리비전에 있던 파일의 내용입니다. 즉 그 파일은 마지막에 편집한 파일의 직전 상태 즉 체크아웃 시점에서의 파일입니다.

filename.rNEWREV

이것은 Subversion 클라이언트 프로그램이 작업 복사를 갱신하면서 서버로부터 받은 파일입니다. 이것은 저장소의 HEAD 리비전에 대응하고 있습니다.

여기서 OLDREV은 .svn 디렉토리에 있는 파일의 리비전 번호이고 NEWREV는 저장소에 있는 HEAD의 리비전 번호입니다.

예를 들어 Sally가 저장소에 있는 sandwich.txt 파일을 변경했다고 합시다. Harry는 자신의 작업 복사에서 그 파일 (sandwich.txt) 파일을 변경하고 커밋합니다. Sally는 커밋하기 전에 작업 카피를 갱신합니다만, 즉 같은 REV를 Sally와 Harry가 수정하고 Harry가 Sally보다 먼저 commit했을때 Sally가 commit를 하려고 할 때 Sally는 다음과 같은 충돌의 보고를 받습니다.

```
$ svn update
C sandwich.txt
Updated to revision 2.
$ ls -l
sandwich.txt
sandwich.txt.mine
sandwich.txt.r1
sandwich.txt.r2
```

이 때 Subversion는 세 개의 임시 파일이 삭제될 때까지 sandwich.txt의 커밋을 허가 **하지 않습니다**.

```
$ svn commit --message "Add a few more things"
svn: A conflict in the working copy obstructs the current operation
svn: commit failed (details follow):
svn: Aborting commit: '/home/sally/svn-work/sandwich.txt' remains in conflict.
```

만약 충돌이 발생했을 경우에는 다음의 세 가지 방법중 한 가지를 할 필요가 있습니다.

- "손으로" 충돌 텍스트를 합칩니다(파일중의 충돌 표지를 보고 편집하여).
- 작업 파일에 임시 파일 중에 하나를 덮쓰기합니다.
- **svn revert filename**를 실행해 로컬에서의 모든 변경을 버립니다.

충돌을 해결하면 **svn resolved**를 실행해서 Subversion에 그것을 알려야 합니다. 이것은 세 개의 임시파일을 삭제해 Subversion은 이제 해당 파일이 충돌 상태에 있다고 생각하지 않게 됩니다. [\[4\]](#)

```
$ svn resolved sandwich.txt
Resolved conflicted state of sandwich.txt
```

1.5.4.1. 충돌을 수동으로 Merge

수동으로 충돌을 합치는 것은 처음에는 매우 싫은 일이겠지만 조금 연습하면 오토바이에서 내리는 것과 같이 간단하게 할 수 있게 됩니다.

예를 들어 당신과 당신의 동료 Sally의 대화 부족으로 인해 두 사람이 sandwich.txt라는 파일을 동시에 편집했다고 합시다. Sally는 자신의 변경을 커밋했고 당신이 작업 카피를 갱신하려고 하면 충돌이 일어납니다. 그래서 충돌을 해결하기 위해 sandwich.txt를 편집해야 합니다. 파일을 보겠습니다.

```
$ cat sandwich.txt
Top piece of bread
Mayonnaise
Lettuce
Tomato
Provolone
. mine
Salami
Mortadella
Prosciutto
=====
Sauerkraut
Grilled Chicken
. r2
Creole Mustard
Bottom piece of bread
```

작다 같다 크다는 기호는 충돌 표지입니다. 앞부분의 두 기호로 둘러싸인 부분은 당신이 고친 것입니다.

```
. mine
Salami
Mortadella
Prosciutto
=====
```

그리고 두 번째와 세 번째의 충돌 표시 사이의 텍스트는 Sally가 커밋한 부분입니다.

```
=====
Sauerkraut
Grilled Chicken
. r2
```

보통 충돌 표시와 Sally의 변경 부분을 그냥 삭제해버리고 싶지는 않을 것입니다. 그런 일을 하면 Sally가 sandwich를 받았을 때에 놀라고 그것은 그녀가 바라는 일은 아닐 것입니다. 당신은 전화를 걸든지 사무실을 건너가서 Sally에게 이탈리아 음식점에서는 소금에 절인 양배추(sauerkraut)를 살 수 없다고 설명해 줍니다(두 명의 변경이 충돌하고 있는 것을 설명합니다). [\[5\]](#) 커밋할 변경 내용에 대해 합의를 했으면 파일을 편집해 충돌 표시를 삭제합니다.

```
Top piece of bread
Mayonnaise
Lettuce
Tomato
Provolone
Salami
Mortadella
```

```
Prosciutto
Creole Mustard
Bottom piece of bread
```

이제 **svn resolved**를 실행하고 커밋합니다.

```
$ svn resolved sandwich.txt
$ svn commit -m "Go ahead and use my sandwich, discarding Sally's edits. "
```

충돌이 있는 파일을 고치는 중에 혼란스럽다면 언제나 Subversion이 당신을 위해서 만든 작업 복사에 있는 세 개의 임시 파일을 참고할 수 있습니다. 그 중에는 갱신전에 당신이 수정한 버전의 파일도 있습니다.

1.5.4.2. 작업 파일 위에 파일을 카피하기

충돌이 일어나자 자신이 한 변경을 무시하려고 할 경우에는 Subversion이 만든 임시 파일 중 하나를 단순히 작업 파일에 덮어쓰면 됩니다.

```
$ svn update
C  sandwich.txt
Updated to revision 2.
$ ls sandwich.*
sandwich.txt  sandwich.txt.mine  sandwich.txt.r2  sandwich.txt.r1
$ cp sandwich.txt.r2 sandwich.txt
$ svn resolved sandwich.txt
$ svn commit -m "Go ahead and use Sally's sandwich, discarding my edits. "
```

1.5.4.3. Punting: svn revert의 이용

충돌이 일어나 조사 결과 자신의 변경을 버리고 편집을 다시 하는 경우는 단지 변경을 원래대로 되돌립니다.

```
$ svn revert sandwich.txt
Reverted sandwich.txt
$ ls sandwich.*
sandwich.txt
```

충돌 파일을 원래대로 되돌릴 때는 **svn resolved**를 실행할 필요가 없는 것에 주의하세요.

이제 당신의 변경을 커밋할 준비가 되었습니다. **svn resolved**는 이 장에서 본 다른 대부분의 커맨드와는 달리 인수를 필요로 합니다. 어떠한 경우에서도 충분히 주의하고 파일중의 충돌을 확실히 해결한 경우만 **svn resolved**를 실행하세요. 임시 파일이 삭제되면 Subversion은 파일이 충돌 마커를 포함하고 있어도 커밋합니다.

1.5.5. 변경점을 커밋

드디어 여기까지 왔습니다. 서버로부터의 변경을 모두 합쳤고 편집은 모두 끝났습니다. 이제 변경을 저장소에 커밋 할 준비가 다 되었습니다.

svn commit 명령은 당신의 모든 변경점을 저장소에 보냅니다. 커밋할 때에는 바뀐점을 설명하는 로그 메시지를 줄 필요가 있습니다. 로그 메시지는 당신이 만든 새로운 리비전에 붙여집니다. 로그 메시지가 간단한 경우는 `--message` (혹은 `-m`) 옵션을 사용해 명령줄에서 지정할 수 있습니다.

```
$ svn commit --message "Corrected number of cheese slices. "
Sending      sandwich.txt
Transmitting file data .
Committed revision 3.
```

그러나 벌써 로그 메시지를 만들어 둔 경우는 `--file` 옵션 뒤에 파일 이름을 지정하여 Subversion이 그 파일의 내용을 사용하도록 지시할 수 있습니다.

```
svn commit --file logmsg
Sending          sandwich
Transmitting file data .
Committed revision 4.
```

`--message`나 `--file` 옵션을 지정하지 않은 경우에는 환경 변수 `$EDITOR`로 지정한 편집기가 떠서 로그 메시지를 작성할 수 있게 해줍니다.

Tip: 만약 편집기에서 로그 메시지를 편집하다가 그 커밋을 중지하고 싶다고 생각했을 경우에는 그저 저장하지 않고 편집기를 종료하면 됩니다. 벌써 커밋 메시지를 저장했다면 내용을 삭제하고 한 번 더 저장해 주세요.

```
$ svn commit
Waiting for Emacs...Done

Log message unchanged or not specified
a) abort, c) continue, e) dit
a
$
```

저장소는 변경점의 내용에 의미가 있을지 어떨지는 전혀 신경쓰지 않습니다. Subversion은 당신이 수정한 파일을 다른 사람이 수정하고 있지 않는 것만을 확인합니다. 만약 다른 사람이 그러한 변경을 **하고 있으면** 커밋은 당신의 변경 한 파일의 어떤 것인가가 최신이 아니라고 하는 메시지를 보내고 커밋은 실패합니다.

```
$ svn commit --message "Add another rule"
Sending          rules.txt
svn: Transaction is out of date
svn: commit failed (details follow):
svn: out of date: `rules.txt' in txn `g'
$
```

이러한 경우에는 **svn update**를 실행하여 그 결과로 나오는 충돌을 해소한 후 한 번 더 커밋해 주세요.

이것으로 Subversion을 사용하는 기본적인 작업 사이클을 설명했습니다. Subversion에는 이 그 밖에도 저장소나 작업 복사를 관리하기 위한 많은 기능이 있습니다만 이 장에서 지금까지 설명해 온 명령만을 사용해도 매우 많은 일을 할 수 있습니다.

1.6. 히스토리 확인하기

이전에 말한 것처럼 저장소는 타임 머신과 같은 곳입니다. 지금까지 커밋된 모든 변경을 기록하기 때문에 파일이나 디렉토리 거기에 딸린 메타데이터의 이전 버전을 보는 것에 의해 히스토리를 조사할 수가 있습니다. Subversion 명령 하나로 과거의 특정 일자나 리비전 번호의 저장소 상태를 체크아웃 (혹은 벌써 있는 작업 카피의 복원) 할 수 있습니다. 그러나 **과거로 돌아가지는** 않고 단지 과거가 어땠는지 **조금 들여다 보기만** 하고 싶은 일도 자주 있습니다.

저장소의 히스토리 데이터를 다루기 위한 명령이 몇 개 있습니다.

svn log

전반적인 정보를 표시합니다. 리비전에 딸린 로그 메시지와 각각의 리비전에서 어느 경로가 변경되었는지를 표시합니다.

svn diff

시간에 따라 파일이 어떻게 변경되어 왔는지를 표시합니다.

svn cat

특정 리비전 번호에서 파일을 추출해 화면에 표시합니다.

svn list

지정한 리비전의 파일이나 디렉토리를 표시합니다.

1.6.1. svn log

파일이나 디렉토리의 히스토리에 관한 정보를 보고 싶을 때는 **svn log** 명령을 사용하세요. **svn log**는 파일이나 디렉토리를 누가 변경했는지 어느 리비전을 변경한 것인지 그 리비전의 시각과 날짜는 언제 인지 그리고 커밋에 딸린 로그 메시지를 표시합니다. 어느 리비전으로 그것이 변경되고 인가 그 리비전의 시각과 일자 한층 더 만약 존재하면 커밋에 부수 한 로그 메시지를 표시합니다.

```
$ svn log
```

```
-----
r3 | sally | Mon, 15 Jul 2002 18:03:46 -0500 | 1 line
```

```
Added include lines and corrected # of cheese slices.
```

```
-----
r2 | harry | Mon, 15 Jul 2002 17:47:57 -0500 | 1 line
```

```
Added main() methods.
```

```
-----
r1 | sally | Mon, 15 Jul 2002 17:40:08 -0500 | 2 lines
```

```
Initial import
-----
```

기본값으로 로그 메시지는 **시간 순서와 반대로** 표시되는 것에 주의하세요. 다른 범위의 리비전을 특정 순서로 보고 싶은 경우나 한 리비전을 찍어서 보고 싶을 때는 **--revision (-r)** 옵션을 사용합니다.

```
$ svn log --revision 5:19      # shows logs 5 through 19 in chronological order
```

```
$ svn log -r 19:5              # shows logs 5 through 19 in reverse order
```

```
$ svn log -r 8                  # shows log for revision 8
```

하나의 파일이나 디렉토리의 로그 히스토리를 볼 수도 있습니다. 예를 들어

```
$ svn log foo.c
```

```
$ svn log http://foo.com/svn/trunk/code/foo.c
```

이것은 작업 파일(또는 URL)이 변경된 리비전 **만을** 표시합니다.

만약 파일이나 디렉토리에 대해 좀 더 상세한 정보를 얻고 싶을 때에는 **svn log**에 **--verbose (-v)** 옵션을 줄 수 있습니다. Subversion은 파일이나 디렉토리를 이동시키거나 복사 할 수 있으므로 파일 시스템의 경로 변화를 쫓을 수 있는 것은 중요합니다. 자세한 출력 상태에서는 **svn log**는 출력 리비전에 서 **변경된 경로** 정보도 포함합니다:

```
$ svn log -r 8 -v
```

```
-----
r8 | sally | 2002-07-14 08:15:29 -0500 | 1 line
```

```
Changed paths:
```

```
U /trunk/code/foo.c
```

```
U /trunk/code/bar.h
```

```
A /trunk/code/doc/README
```

```
Frozzled the sub-space winch.
```


왜 svn log가 아무것도 안보여주지?

Subversion을 사용한지 얼마 지나지 않았을 때 대부분의 사용자는 다음과 같은 일을 당하겠지요.

```
$ svn log -r 2
```

```
$
```

얼핏보기에 에러 같습니다만 리비전은 저장소에 대한 것이므로 **svn log**는 저장소 경로 별로 동작하는 것을 생각하세요. (경로를 지정하지 않으면 Subversion은 기본값으로 ". ."을 사용합니다) 그 결과 작업 카피의 하위 디렉토리를 조작하고 있기 때문에 그 디렉토리 아래에 있는 파일에는 아무것도 변경된 것이 없는데 로그를 보려고 하면 Subversion은 아무것도 출력하지 않습니다. 그 리비전에서의 변경을 보고 싶으면 작업 카피의 최상위에서 실행하세요.

1.6.2. svn diff

svn diff는 앞에서 다룬 적이 있습니다. 이것은 unified diff 형식으로 파일의 차이점을 표시합니다. 저장소에 커밋하기 전에 작업 카피에서 변경된 점을 표시하는데 사용할 수 있습니다.

사실 **svn diff**에는 **세 가지** 사용법이 있습니다.

- 로컬 변경 내용을 확인하기
- 작업 카피와 저장소를 비교
- 저장소와 저장소를 비교

1.6.2.1. 로컬의 변경 내용을 확인하기

이미 봐 온 것처럼 옵션없이 **svn diff**를 실행하면 작업 카피의 내용과 .svn에 캐쉬된 "수정된 리비전"의 을 비교합니다.

```
$ svn diff
Index: rules.txt
=====
--- rules.txt      (revision 3)
+++ rules.txt      (working copy)
@@ -1, 4 +1, 5 @@
 Be kind to others
 Freedom = Responsibility
 Everything in moderation
 -Chew with your mouth open
 +Chew with your mouth closed
 +Listen when others are speaking
 $
```

1.6.2.2. 작업 카피와 저장소를 비교하기

--revision(-r)를 하나만 지정하면 작업 카피는 저장소의 지정된 리비전과 비교 됩니다.

```
$ svn diff --revision 3 rules.txt
Index: rules.txt
=====
--- rules.txt      (revision 3)
+++ rules.txt      (working copy)
@@ -1, 4 +1, 5 @@
```

```

Be kind to others
Freedom = Responsibility
Everything in moderation
-Chew with your mouth open
+Chew with your mouth closed
+Listen when others are speaking
$

```

1.6.2.3. 저장소와 저장소를 비교하기

--revision(-r)에 콜론으로 구분된 리비전 번호 두 개를 지정하면 두 개의 저장소가 비교됩니다.

```

$ svn diff --revision 2:3 rules.txt
Index: rules.txt
=====
--- rules.txt      (revision 2)
+++ rules.txt      (revision 3)
@@ -1, 4 +1, 4 @@
  Be kind to others
-Freedom = Chocolate Ice Cream
+Freedom = Responsibility
  Everything in moderation
  Chew with your mouth closed
$

```

svn diff는 작업 카피에 있는 파일을 저장소와 비교할 때만 쓸 수 있는것이 아닙니다. 작업 카피가 없어도 URL 인수를 주는 것으로 저장소에 있는 아이템과의 차이를 조사할 수가 있습니다. 이것은 로컬 머신에 작업 카피가 없을 때에 파일의 변경점을 알고 싶은 경우에 매우 편리합니다.

```

$ svn diff --revision 4:5 http://svn.red-bean.com/repos/example/trunk/text/rules.txt
$

```

1.6.3. svn cat

만약 이전 버전의 파일을 보고 싶지만 두 파일 사이의 차이를 볼 필요는 없는 경우에는 **svn cat**를 사용할 수 있습니다.

```

$ svn cat --revision 2 rules.txt
Be kind to others
Freedom = Chocolate Ice Cream
Everything in moderation
Chew with your mouth closed
$

```

직접 파일에 출력할 수도 있습니다.

```

$ svn cat --revision 2 rules.txt > rules.txt.v2
$

```

아마도 어째서 낡은 리비전으로 되돌리기 위해서 단순히 **svn update --revision**을 사용하지 않는 것인지 궁금할 것입니다. **svn cat**를 사용하는 편이 좋은 이유가 몇 가지 있습니다.

우선 외부의 diff(GUI이거나 unified diff 형식의 출력이 아무런 의미도 없는 경우) 프로그램으로 두 리비전의 차이점을 보고 싶을지도 모릅니다. 이 경우 낡은 버전의 내용을 얻어서 파일에 출력한 것과 작업 카피중의 파일을 외부 diff 프로그램에 전해줍니다.

자주 다른 리비전과의 차이점을 취하는 것보다는 그 버전의 파일 전체를 보는 편이 간단한 일이 있습니다.

1.6.4. svn list

svn list 명령은 로컬 머신에 실제로 파일을 받지 않고도 저장소에 어떤 파일이 있는지 보여줍니다.

```
$ svn list http://svn.collab.net/repos/svn
README
branches/
clients/
tags/
trunk/
```

좀 더 자세한 표시를 원한다면 **--verbose (-v)** 옵션을 지정합니다. 출력은 다음과 같이 됩니다.

```
$ svn list --verbose http://svn.collab.net/repos/svn
2755 harry          1331 Jul 28 02:07 README
2773 sally          Jul 29 15:07 branches/
2769 sally          Jul 29 12:07 clients/
2698 harry          Jul 24 18:07 tags/
2785 sally          Jul 29 19:07 trunk/
```

각각의 세로줄은 파일 또는 디렉토리가 마지막으로 바뀐 리비전 수정한 사람, 파일인 경우 그 파일의 크기, 마지막으로 바뀐 날짜, 그리고 이름입니다.

1.6.5. 히스토리 기능에 대한 마지막 말

지금까지 보아온 모든 명령에 더해서 **svn update**와 **svn checkout** 에 **--revision** 옵션을 주어서 실행할 수 있습니다. 이것은 작업 카피 전체를 "과거 시점"으로 되돌립니다. [\[6\]](#):

```
$ svn checkout --revision 1729 # Checks out a new working copy at r1729
```

```
$ svn update --revision 1729 # Updates an existing working copy to r1729
```

1.7. 자주 사용되는 다른 명령들

앞에서 본 것들만큼 자주 이용되는 것은 아니지만 다음의 명령이 가끔 필요하게 됩니다.

1.7.1. svn cleanup

Subversion이 작업 카피(또는 .svn에 있는 정보)를 수정할 경우에는 가능한 안전하게 하려고 합니다. 무엇인가를 변경하기 전에 실행 내용을 로그 파일에 쓰고 그 명령을 실행한 후 마지막에 로그 파일을 삭제합니다(이것은 저널링 파일 시스템의 방식과 비슷합니다). Subversion의 동작이 중단되면 (Control-C 를 입력하거나 컴퓨터가 종료되었을 경우) 로그 파일은 디스크에 남습니다. 로그 파일을 재실행하는 것으로 Subversion은 이전에 하던 작업을 완료할 수 있고 작업 카피는 정상적인 상태로 되돌아갑니다.

svn cleanup가 하는 일은 정확히 이런 것입니다. 작업 카피를 뒤져서 남은 로그를 실행하고 프로세스의 잠금을 없앱니다. 작업 카피의 어딘가가 "잠겨" 있다고 Subversion이 알려줄 때 이 명령을 실행하세요. **svn status**는 잠겨있는 항목 옆에 L을 표시합니다.

```
$ svn status
L    . /somedir
M    . /somedir/foo.c
```

```
$ svn cleanup
$ svn status
M      . /somedir/foo.c
```

1.7.2. svn import

import 명령은 버전 관리되고 있지 않은 여러파일들을 저장소로 이동하는 간단한 방법입니다.

```
$ svnadmin create /usr/local/svn/newrepos
$ svn import mytree file:///usr/local/svn/newrepos/fooproject
Adding mytree/foo.c
Adding mytree/bar.c
Adding mytree/subdir
Adding mytree/subdir/quux.h
Transmitting file data....
Committed revision 1.
```

위의 예는 디렉토리 mytree의 내용을 저장소의 fooproject 디렉토리에 넣습니다.

```
/fooproject/foo.c
/fooproject/bar.c
/fooproject/subdir
/fooproject/subdir/quux.h
```

1.8. 요약

이것으로 Subversion 클라이언트의 명령 대부분에 대해서 설명했습니다. 여기서 설명하지 않았지만 중요한 것은 브랜치(branch)와 병합(merge) (> 참조) 그리고 속성입니다 (> 참조). Subversion이 가지고 있는 많은 명령을 파악하려면 그리고 여러분의 일을 쉽게 하기 위해서 그것들을 어떻게 사용하는지 알아보려면 >를 훑어보는 것이 좋을것입니다.

Notes

[1]

작업본에 있는 디렉토리는 모두 .svn이라는 하위 디렉토리를 포함하고 있다는 사실은 제외하고 말합니다. 나중에 좀 더 이야기 하지요.

[2]

물론 저장소(repository)로부터 완전하게 삭제되어 버리는 것은 아닙니다. 단지 저장소(repository)의 HEAD로부터 삭제될 뿐입니다. 삭제한 리비전보다 전의 리비전을 지정해 체크 아웃 하면(혹은 작업 복사를 갱신하면) 삭제전 상태로 돌아올 수가 있습니다.

[3]

Subversion은 내부 diff 엔진을 이용해서 unified diff 형식을 생성합니다. 만약 다른 형식의 diff 출력을 갖고 싶은 경우에는 --diff-cmd로 외부 diff 프로그램을 지정하고 --extensions 옵션을 사용해 플래그를 건네주세요. 예를 들어 파일 foo.c의 로컬 변경점을 context 출력 형식에서 보고 싶지만 공백의 변경은 무시하고 싶은 경우 "svn diff --diff-cmd /usr/bin/diff --extensions '-bc' foo.c" 와 같이 실행할 수 있습니다.

[4]

당신은 임시 파일을 항상 직접 삭제할 수가 있습니다. 하지만 Subversion이 당신을 위해 명령을 준비하고 있는데 정말로 그런일을 하고 싶으십니까? 그렇게는 생각되지 않습니다만.

[5]

그리고 당신이 주문하면 그들은 당신을 기차에 태워서 마을의 밖으로 보내버릴 지도 몰라요.

[6]

알겠습니까? Subversion은 타임 머신이라고 말했죠?



Last modified: 2005-04-04 01:58:20, Processing time: 0.0474 sec

[인터넷 익스플로러 7, 무료](#)

Google에 최적화. 향상된 보안 기능. 오늘 다운로드 받으세요!

[Subversion Made Better](#)

Transform subversion into a multi site development solution